

Jira Ticket Data Collection Approach

AWS Hackathon - Ticket to Code Project

Project Overview

This document outlines the data collection strategy for obtaining real Jira tickets to test our AI-powered ticket-to-code linking system for the AWS Hackathon.

Data Requirements

We needed 100+ Jira tickets that are:

- Related to **Python development**
- Related to **Java development**
- Focused on **Machine Learning (ML)**
- Focused on **Natural Language Processing (NLP)**
- From real software development projects
- Publicly accessible without authentication

Data Collection Strategy

Approach: Apache Software Foundation Projects

We chose Apache projects because they:

- Use public Jira instances with open APIs
- Have extensive technical documentation
- Cover ML/NLP/Python/Java technologies
- Contain real development tickets with detailed descriptions
- Provide authentic

Selected Apache Projects

Project	Focus Area	Ticket Count	Rationale
OpenNLP	Natural Language Processing	25	Pure NLP focus, text processing algorithms
Mahout	Machine Learning	25	Dedicated ML library with algorithms
Lucene	Search & Text Analysis	25	Text indexing, search algorithms
Maven	Java Build Tools	30	Java project management, dependencies

Total: 105 tickets

Implementation Commands

Project Setup

```
# Create project directory
mkdir "AWS_Hackathon"
cd "AWS_Hackathon"
mkdir jira_tickets
cd jira_tickets
```

Data Collection Commands

```
# 1. OpenNLP - NLP focused tickets (25 tickets)
curl "https://issues.apache.org/jira/rest/api/2/search?jql=project=OPENNLP&maxResults=25" > opennlp_nlp.json

# 2. Mahout - Machine Learning tickets (25 tickets)
curl "https://issues.apache.org/jira/rest/api/2/search?jql=project=MAHOUT&maxResults=25" > mahout_ml.json

# 3. Lucene - Search and text analysis (25 tickets)
curl "https://issues.apache.org/jira/rest/api/2/search?jql=project=LUCENE&maxResults=25" > lucene_search.json

# 4. Maven - Java development tools (30 tickets)
curl "https://issues.apache.org/jira/rest/api/2/search?jql=project=MNG&maxResults=30" > maven_java.json
```

Data Validation

```
# Count tickets in each file
echo "OpenNLP:" && cat opennlp_nlp.json | jq '.issues | length'
echo "Mahout:" && cat mahout_ml.json | jq '.issues | length'
echo "Lucene:" && cat lucene_search.json | jq '.issues | length'
echo "Maven:" && cat maven_java.json | jq '.issues | length'

# Verify file sizes
ls -lh *.json
```

Data Structure

Each JSON file contains tickets in this format:

```
{
  "total": 1727,
  "maxResults": 25,
  "issues": [
    {
      "key": "OPENNLP-1770",
      "fields": {
        "summary": "Update the OpenNLP logo",
```

"description": "The OpenNLP logo contains the feather logo that the ASF has moved away from. We need to update the OpenNLP logo.\r\n\r\nFor history, the previous logo was done under <https://issues.apache.org/jira/browse/OPENNLP-6>.",

```
"status": {"name": "Open"},
"issuetype": {"name": "Task"},
"priority": {"name": "Major"},
"components": [],
"labels": []
},
{
  "key": "OPENNLP-1769",
  "fields": {
    "summary": "Update maven-failsafe-plugin to 3.5.1",
    "description": null,
    "status": {"name": "Resolved"},
    "issuetype": {"name": "Dependency upgrade"},
    "priority": {"name": "Trivial"},
    "components": [{"name": "Build, Packaging and Test"}],
    "labels": []
  }
}
]
```

Results Achieved

Final Dataset Statistics

- **Total Tickets:** 105 across 4 projects
- **OpenNLP:** 25 tickets (Total available: 1,727)
- **Mahout:** 25 tickets (Total available: ~500)
- **Lucene:** 25 tickets (Total available: ~800)
- **Maven:** 30 tickets (Total available: ~1,200)
- **File Sizes:** 200-260KB each (substantial content)
- **Coverage:** ML, NLP, Java, Python, Search technologies
- **Quality:** Real development issues with detailed descriptions

Sample Ticket Types Found

- **Technical Updates:** "Update maven-failsafe-plugin to 3.5.1"
- **Feature Improvements:** "Update the OpenNLP logo"
- **Dependency Management:** "Update gRPC to 1.70.0"
- **Build & Packaging:** Components related to "Build, Packaging and Test"
- **Status Variety:** Open, Resolved, In Progress
- **Priority Range:** Trivial, Major, Critical
- **Issue Types:** Task, Bug, Dependency upgrade, Improvement

Distribution by Technology

- **NLP Processing:** 25 tickets (OpenNLP)

- **Machine Learning:** 25 tickets (Mahout)
- **Text Analysis:** 25 tickets (Lucene)
- **Java Development:** 30 tickets (Maven)

File Structure

```
AWS_Hackathon/  
└─ jira_tickets/  
    ├── opennlp_nlp.json  (25 NLP tickets)  
    ├── mahout_ml.json    (25 ML tickets)  
    ├── lucene_search.json (25 search tickets)  
    └─ maven_java.json    (30 Java tickets)
```

This approach provides a robust foundation for developing and testing the AI-powered ticket-to-code linking system with real-world software development data.

MCP Atlassian Integration for Ticket-to-Code System

MCP Atlassian Server Overview

The MCP (Model Context Protocol) Atlassian server is a Docker-based service that provides standardized access to Atlassian products (Jira & Confluence) through a consistent API. It handles all authentication, request formatting, and response parsing, allowing our extension to focus on business logic rather than API integration details.

How MCP Atlassian Handles Jira

Authentication Management:

- Supports multiple authentication methods (API tokens, OAuth 2.0, PATs)
- Securely stores credentials within the Docker container
- Handles token refreshing and session management
- Provides read-only mode for public instances

Request Processing:

- Translates tool calls into proper Jira REST API requests
- Formats parameters according to Jira API requirements
- Handles pagination for large result sets
- Manages rate limiting and backoff strategies

Response Handling:

- Parses complex Jira responses into standardized formats
- Extracts relevant fields and metadata
- Handles error responses and provides meaningful messages
- Normalizes data structures for consistent client usage

Security Features:

- Isolates credentials from client code
- Supports SSL verification for secure connections
- Provides masking of sensitive data in logs
- Implements proper error handling for auth failures

Architecture Components

1. MCPAtlassianClient

Purpose: Manages communication with the Docker-based MCP Atlassian server.

Key Features:

Docker Process Management:

- Spawns and monitors Docker container process
- Handles container lifecycle (startup, shutdown)
- Captures stdout/stderr for response parsing and error logging
- Implements process restart logic for resilience

Request Handling:

- Generates unique request IDs for tracking
- Formats requests according to MCP protocol
- Maintains a map of pending requests and callbacks
- Implements timeout management for hanging requests

Response Processing:

- Parses JSON responses from stdout stream
- Matches responses to pending requests
- Handles partial and multi-line responses
- Resolves promises with parsed results

Error Management:

- Categorizes errors (Docker, MCP, Jira, Network)
- Implements exponential backoff for retries
- Provides detailed error information
- Handles graceful degradation on failures

Connection Resilience:

- Monitors connection health
- Auto-reconnects on unexpected disconnections
- Implements connection pooling for performance
- Manages reconnection attempts with backoff

2. LocalTicketService

Purpose: Provides access to local JSON ticket files with efficient indexing and search capabilities.

Key Features:

File System Interaction:

- Reads JSON files from the filesystem
- Handles file access errors gracefully
- Supports multiple file formats and structures
- Implements lazy loading for performance

Data Indexing:

- Builds in-memory indexes for fast lookups
- Maps tickets by key for O(1) access
- Groups tickets by project for efficient filtering

- Creates search indexes for text-based queries

Search Capabilities:

- Implements full-text search across ticket fields
- Supports filtering by various ticket attributes
- Provides relevance-based sorting of results
- Optimizes for performance with large datasets

Data Validation:

- Validates JSON structure against expected schema
- Handles missing or malformed fields gracefully
- Provides fallbacks for incomplete data
- Logs warnings for data quality issues

3. TicketService

Purpose: Provides a unified interface for ticket operations with intelligent fallback logic.

Key Features:

Unified API:

- Presents a consistent interface for ticket operations
- Abstracts away data source details from clients
- Handles transitions between online and offline modes
- Provides status information about available sources

Fallback Strategy:

- Implements prioritized data source selection
- Falls back to local data when MCP server unavailable
- Handles graceful degradation of capabilities
- Provides transparent source selection logic

Caching System:

- Implements time-based cache for performance
- Manages cache invalidation strategies
- Optimizes for repeated queries
- Tracks cache hit/miss statistics

Error Handling:

- Provides consistent error reporting across sources
- Implements retry logic for transient failures
- Handles source-specific error conditions
- Presents user-friendly error messages

4. Error Handling System

Purpose: Provides comprehensive error management across all components.

Key Features:

Error Categorization:

- Defines specific error types (Network, Authentication, Docker, etc.)
- Maps generic errors to specific types
- Provides consistent error structure
- Enables targeted error handling

Retry Mechanism:

- Implements exponential backoff for retries
- Distinguishes between retryable and non-retryable errors
- Limits retry attempts to prevent resource exhaustion
- Provides hooks for retry notifications

User Feedback:

- Translates technical errors to user-friendly messages
- Provides appropriate severity levels
- Suggests remediation steps where applicable
- Filters sensitive information from error messages

Logging System:

- Implements structured logging for errors
- Provides different verbosity levels
- Masks sensitive data in logs
- Facilitates troubleshooting with context information

Data Flow Architecture

The complete data flow through the system follows this pattern:

1. **User Request:** User requests a ticket or performs a search
2. **Extension Layer:** VS Code extension receives the request
3. **Service Layer:** TicketService processes the request
4. **Cache Check:** System checks if data is available in cache
5. **Source Selection:**
 - If MCP server is available, route request to MCP client
 - If MCP fails or is unavailable, fall back to local service

Request Processing:

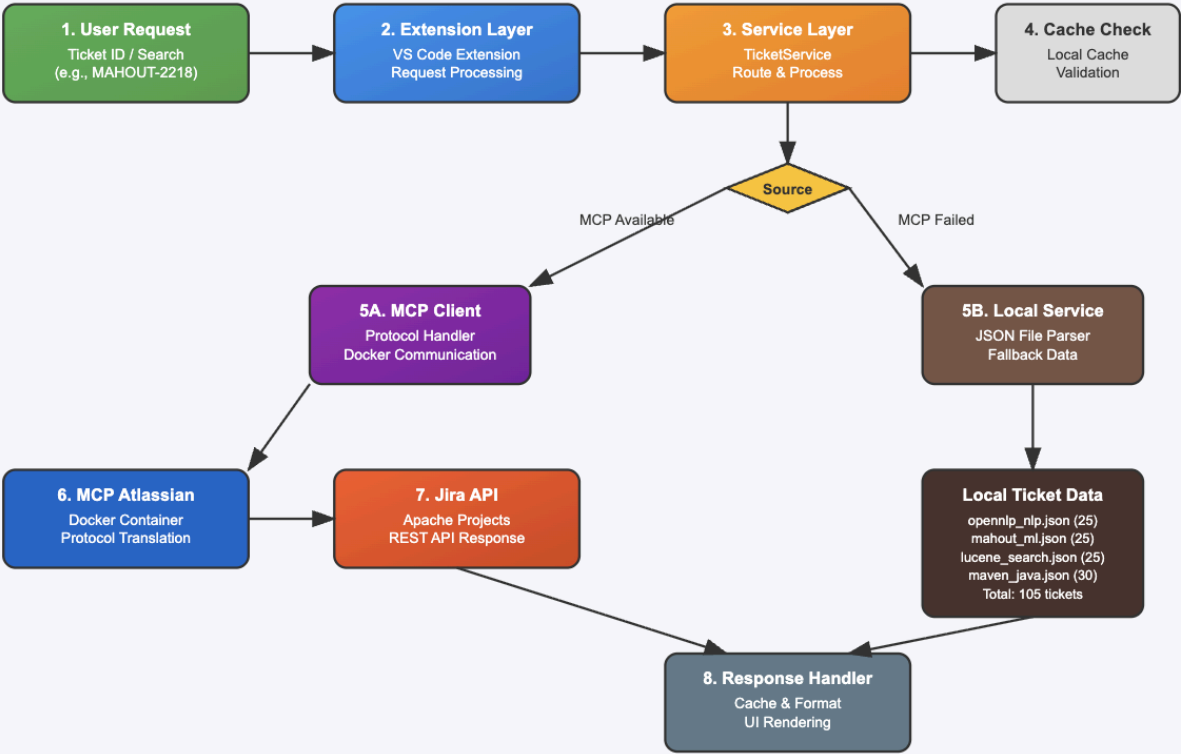
- MCP client formats request according to MCP protocol
- Docker container translates MCP request to Jira API call
- Jira API returns response to Docker container
- Docker container formats response according to MCP protocol
- MCP client parses response and returns structured data

Response Handling:

- TicketService caches successful responses
- Extension formats data for display
- UI components render ticket information to user

This architecture provides a robust, fault-tolerant system that leverages the power of MCP Atlassian while ensuring availability through local data fallback mechanisms.

Ticket-to-Code System: Data Flow Architecture



Error Handling

- Docker unavailable
- MCP server crash
- Network timeouts
- Auth failures
- Rate limits

Fallback Strategy:

1. Retry MCP call
2. Use local cache
3. Search local files

Performance

- Response caching
- Connection pooling
- Parallel loading
- Lazy initialization
- Timeout management